

Exploratory Data Analysis

Dr. Krishna Bathula

Learning Objectives

1. Introduction to EDA
2. Data Understanding and Cleaning
3. Descriptive statistics and Data Summarization
4. Data Visualization Techniques
5. Data Relationships and Correlation
6. Feature Engineering and Transformation

Introduction to EDA

The **purpose** of introducing **Exploratory Data Analysis (EDA)** in the context of data science is to provide a foundational understanding of how to **explore** and **interpret raw data**.

- How to explore and interpret raw data before applying any statistical models or machine learning algorithms
- How does EDA fit into the overall data science workflow?
- How does it help prepare data for advanced analytics and modeling tasks?

Understand the practical application of EDA.

Data and Analysis

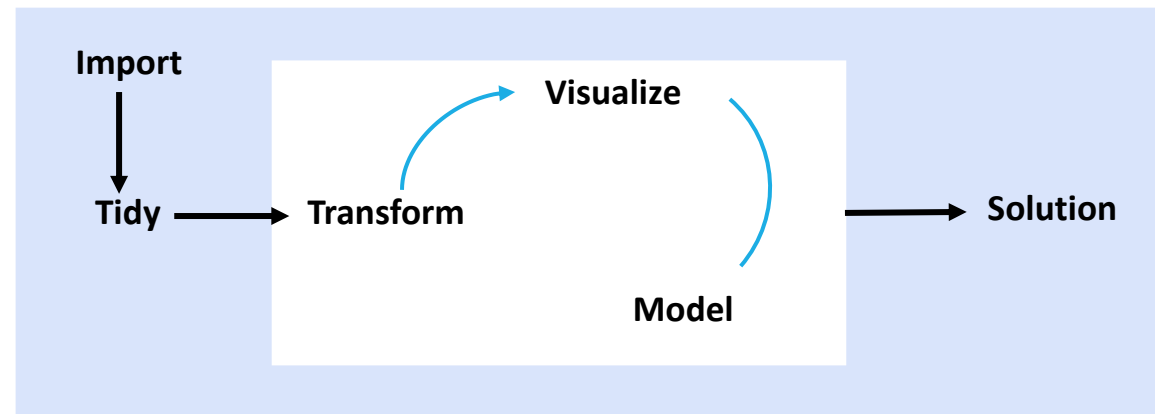
EDA is the first step in the Data Analysis Pipeline. Helps to:

1. Understand the structure and properties of the data:

- This includes the data types, distributions, missing values, and outliers.

2. Identify patterns and relationships:

- EDA helps in discovering underlying trends, correlations, or anomalies that could influence the outcomes of further analysis.



3. Prepare data for modeling:

- It helps to clean and transform data, ensuring that it's suitable for applying more complex models, reducing errors, and improving results.

EDA - Process Steps

Detecting data inconsistencies in a predictive modeling project could save time and improve model accuracy.

Significant steps in the EDA process:

1. **Data Cleaning:** Handling missing values, outliers, and duplicates.
2. **Data Summarization:** Using descriptive statistics to understand data distribution.
3. **Visualization:** Plotting data to find patterns, trends, and anomalies.
4. **Feature Engineering:** Creating new features to better represent data for modeling.

EDA - Tools

Python Libraries

1. Pandas – Data manipulation and summarization
2. Matplotlib/Seaborn – Visualization
3. NumPy/SciPy – Statistical analysis.
4. Scikit-learn



Understanding Datasets

Effective data understanding and cleaning are essential for ensuring the quality and reliability of the data before moving to more advanced stages of the project.

Understand:

1. **Features** (Variables/Columns)
2. **Data Points**(Observations/Rows)
3. **Metadata** - Descriptions of each column's purpose, type (e.g., **numerical**, **categorical**), and role (e.g., **dependent** or **independent variable**).
4. **Basic summary** of the data, including the **count** of records, **mean**, **min**, **max**, and **standard deviation** for numerical columns.

Understanding Datasets

Understand:

5. **Numerical Data:** Data that represents numbers, either continuous (e.g., age, salary) or discrete (e.g., count of items).
 6. **Categorical Data:** Non-numeric data that represents categories or labels (e.g., gender, product category).
 7. **Text Data:** Unstructured data in the form of text (e.g., user reviews, social media posts).
 8. **Missing Values:** Identify which columns or rows contain missing values using `.isnull()` in Python.
-

Understanding Datasets

Understand:

9. **Not a Number (NaN):** A numeric data type that represents an undefined or unrepresentable value.
 - Ex: Infinity, The result of dividing by zero, A missing value, and The square root of a negative number.
10. **Outliers:** Detect extreme values that fall outside the expected range using visual tools like box plots or statistical methods (e.g., Z-scores).

Handle Missing Data

Missing data and outliers may affect the analysis.

Adopt strategies such as deletion and imputation.

1. Deletion:

- Remove rows or columns with missing data if the amount of missing data is small.
- Only use when missing values are sparse deleting them doesn't skew results.

2. Imputation:

- Replace missing data with reasonable estimates, such as the mean, median, or mode for numerical data, or using predictive models.
- Use when missing values are significant, and removing them could lead to biased results.

Data - Statistics

Descriptive statistics help summarize large datasets into meaningful metrics.

Focuses on measures of central tendency (mean, median, mode) and measures of variability (range, variance, standard deviation).

These statistics give a quick sense of the **data's distribution and spread**.

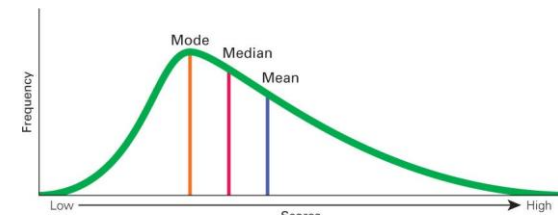
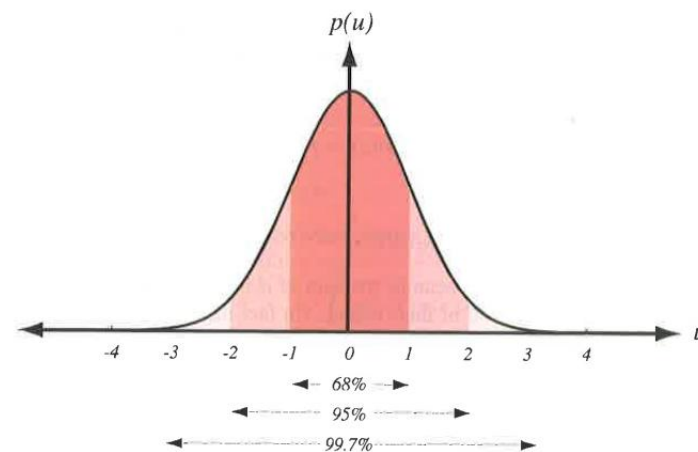
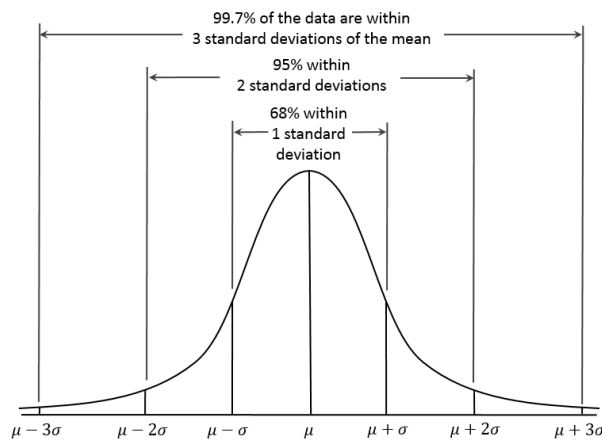
MEAN	MEDIAN	MODE
Find the mean for the data set: 10, 8, 5, 5, 6, 7, 5, 11, 4, 9, 3	Find the median for the data set: 10, 8, 5, 5, 6, 7, 5, 11, 4, 9, 3	Find the median for the data set: 10, 8, 5, 5, 6, 7, 5, 11, 4, 9, 3
Mean = $\frac{\text{total}}{\text{number of values}}$	The median is the middle number.	The mode is the number that has the highest frequency in the data values. Putting the numbers in order from least to greatest is helpful.
Mean = $\frac{10 + 8 + 5 + 5 + 6 + 7 + 5 + 11 + 4 + 9 + 3}{11}$	List the numbers in order from least to greatest: 3 4 5 5 6 7 8 9 10 11	3 4 5 5 5 6 7 8 9 10 11
Mean = $\frac{73}{11}$	Median is 6.	Mode is 5.
Mean = 6.64 (rounded to 2 decimal places)		

- **Mean:** The arithmetic average. **Sensitive to outliers.**
- **Median:** The middle value of a dataset when ordered, providing a robust measure of central tendency in **skewed distributions**.
- **Mode:** The most frequently occurring value in a dataset, useful for **categorical data**.

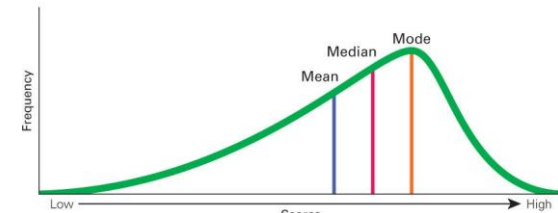
Data - Distributions

A distribution shows how frequently values occur in the dataset, providing a sense of whether the data is symmetric, skewed, or has outliers.

Common distributions include normal distributions, where data follows a bell curve, and skewed distributions, where data is lopsided.



(a) Right-skewed distribution



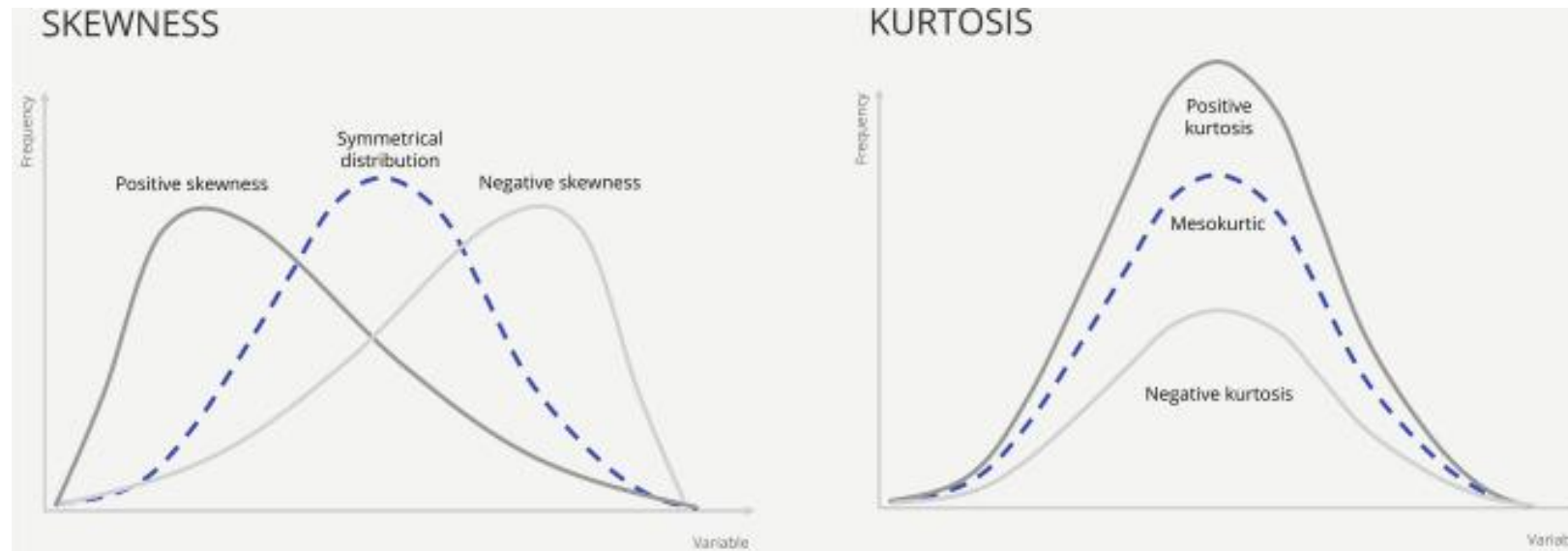
(b) Left-skewed distribution

Visualizing distributions (e.g., using histograms) helps in identifying patterns like skewness, multimodal distributions, or outliers.

Data – Skewness & Kurtosis

1. **Skewness** - Measures the asymmetry of the data distribution.

- **Positive Skewness:** The Right tail is longer. The mean is greater than the median.
- **Negative Skewness:** The Left tail is longer. The mean is less than the median.



2. **Kurtosis** - Measures the "tailedness" or how sharply peaked a distribution is.

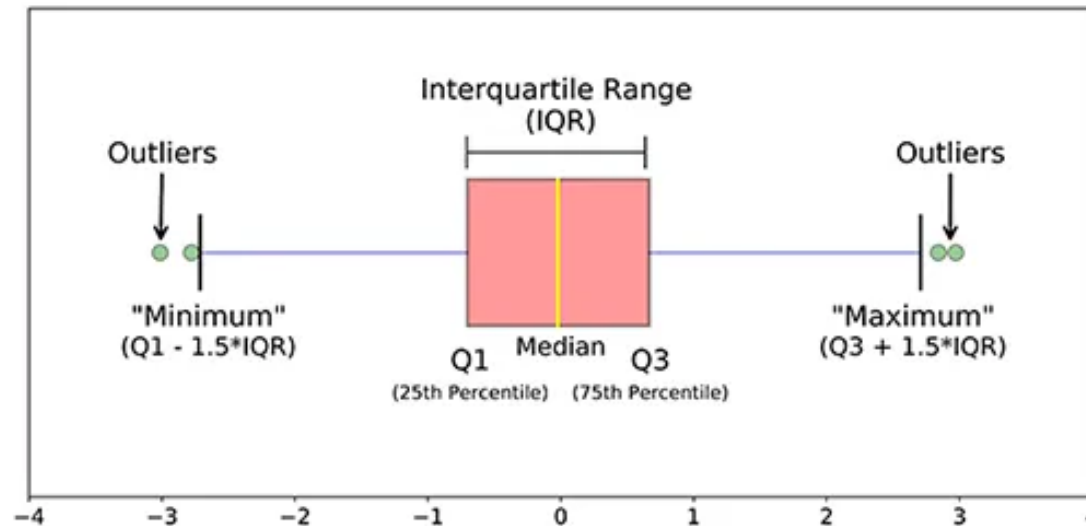
- High Kurtosis: More outliers (longer tails).
- Low Kurtosis: Fewer outliers (shorter tails).

Handle Outliers

Handle extreme data points that can distort results.

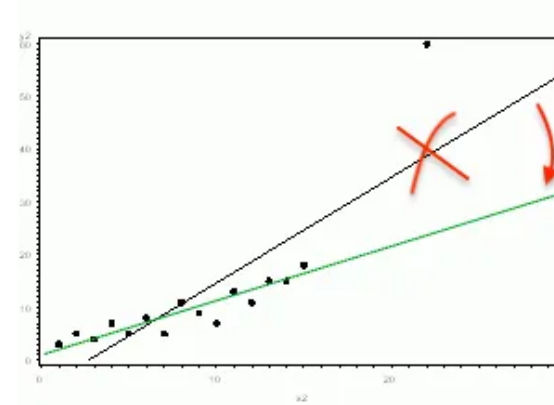
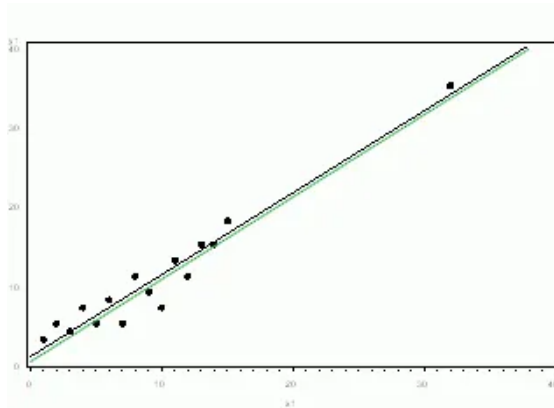
Identify Outliers with statistical methods like Interquartile Range, Z-scores.

Use visualizations like histograms or box plots to identify potential outliers in a dataset and missing value checks using Pandas.

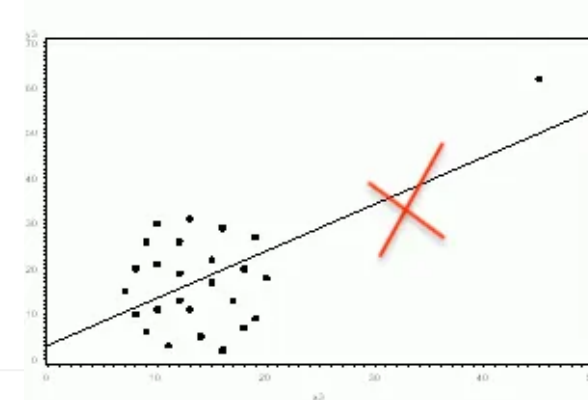


Handle Outliers

Remove them if they are, likely to be errors.



Retain if they represent valid but rare events.



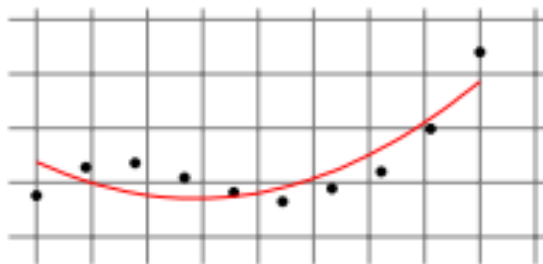
Transform (log transform) if they follow a non-normal distribution.

Outlier - Caution

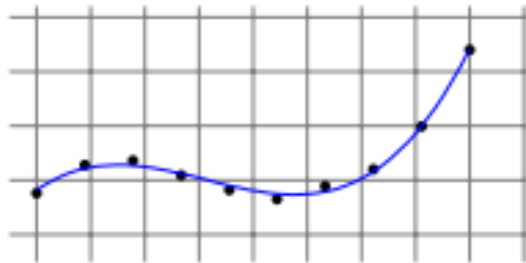
Understand if the model may lean toward polynomial regression with non-linear datasets.

Ex: In Polynomial Regression, the model adapts to the best fit, not with a straight line but a **curved line**, and involves **feature scaling**.

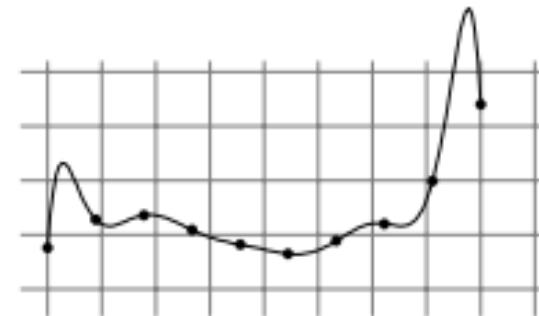
degree 2



degree 3



degree 10



Handle Duplicates

Duplicate records can distort analysis, leading to misleading results.

- Removing duplicate rows in a dataset ensures data integrity.

	A	B	C
1	1	7	
2	2	8	
3	3	9	
4	4	9	
5	5	1	
6	6	10	
7		10	
8			
9			

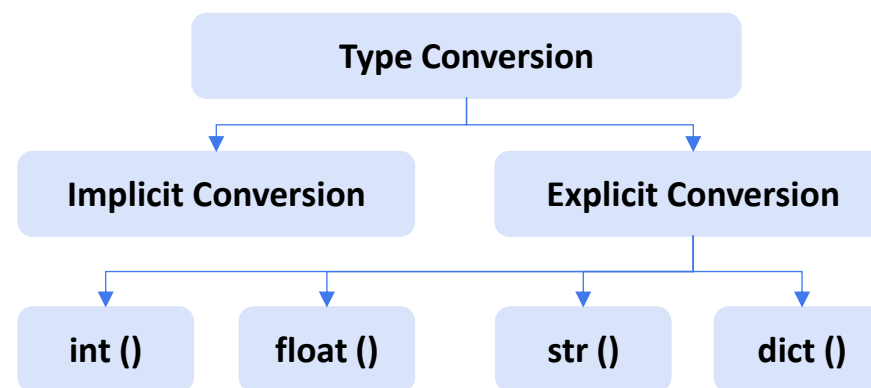


- Use methods like `.drop_duplicates()` in Pandas to remove duplicates based on specific columns or across the entire dataset.

Data Type Conversions

Sometimes data is incorrectly formatted (e.g., numerical data stored as text)

- If this is the case, converting the data to the correct type (e.g., int, float, or string) is essential.
- Converting categorical data to numerical
- Converting text data to a suitable format (e.g., tokenization for natural language processing).
- Use the `.astype()` function in Pandas to demonstrate converting a column from one data type to another.



Data - Standardize and Normalize

1. **Standardization:** Scales data so that it has a mean of 0 and a standard deviation of 1
 - When using algorithms sensitive to data scales such as SVMs or K-Means.
2. **Normalization:** Rescales data between 0 and 1
 - Use this when using an algorithm that expects input in a specific range (ex: image pixels read by Neural Networks)



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	29	150	195	254	255	254	176	193	150	96	0	0	0
2	0	0	0	48	166	224	253	253	234	196	253	253	253	253	233	0	0	0
3	0	93	244	249	253	187	46	10	8	4	10	194	253	253	233	0	0	0
4	0	107	253	253	230	48	0	0	0	0	0	192	253	253	156	0	0	0
5	0	3	20	20	15	0	0	0	0	0	43	224	253	245	74	0	0	0
6	0	0	0	0	0	0	0	0	0	0	249	253	245	126	0	0	0	0
7	0	0	0	0	0	0	14	101	223	253	248	124	0	0	0	0	0	0
8	0	0	0	0	0	11	166	239	253	253	253	187	30	0	0	0	0	0
9	0	0	0	0	0	16	248	250	253	253	253	253	232	213	111	2	0	0
10	0	0	0	0	0	0	43	98	98	208	253	253	253	253	187	22	0	0

Data Visualization

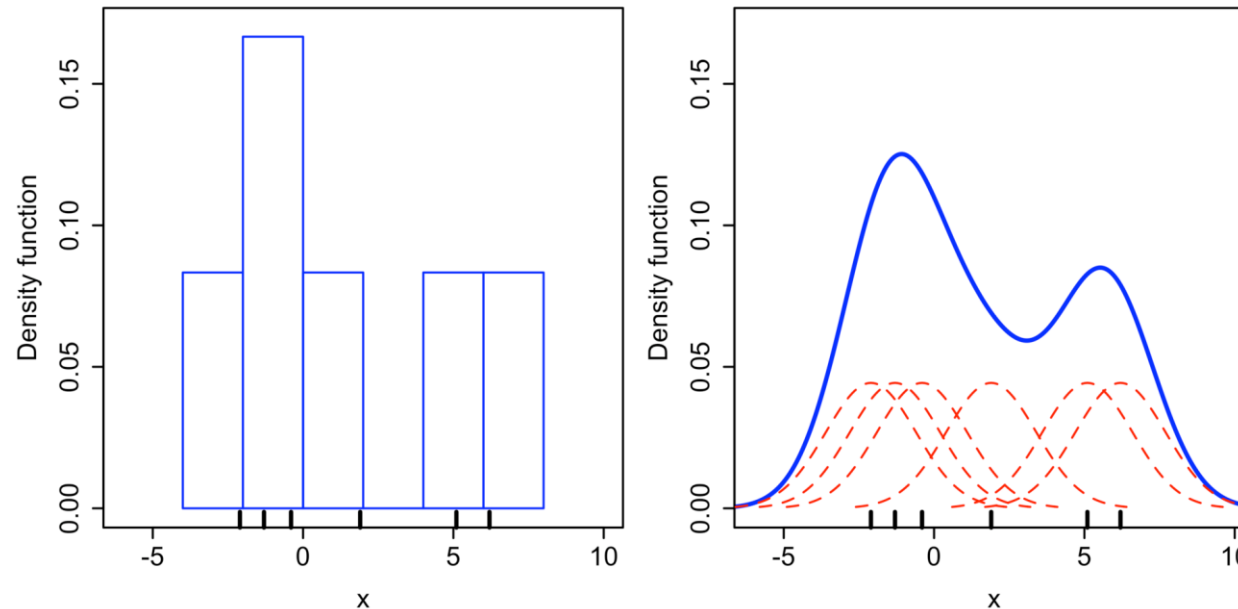
Visualizations provide an intuitive way to **explore data**, **uncover patterns**, spot **anomalies**, and **gain insights** before applying statistical models.

- Different types of plots are used to illustrate various aspects of the data, such as **distributions**, **relationships**, and **proportions**.
- Visuals help simplify complex data and reveal insights that are hard to spot in raw data.
- **Patterns**, **correlations**, and **outliers** can be quickly identified.
- Allows stakeholders to intuitively understand data insights, especially non-technical audiences.

Data Visualization - Types

1. Distributions: Histograms, KDE

- **Histograms:** Show the frequency distribution of numerical data by dividing data into bins and counting the number of observations in each bin.

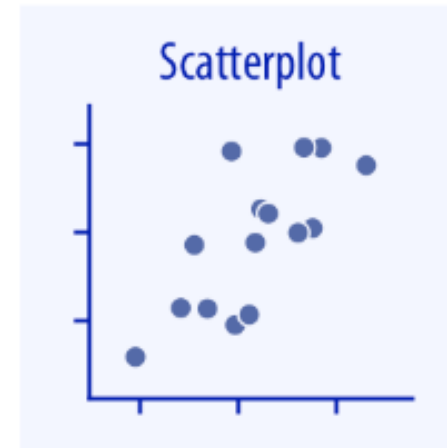
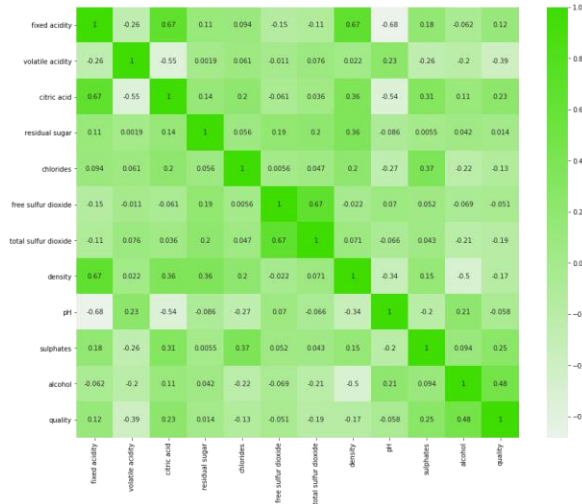


- **KDE Plots** (Kernel Density Estimation): A smoothed version of the histogram that shows the probability density function of a variable.

Data Visualization - Types

2. Relationships: Correlation heat maps, Scatter plots, and joint plots.

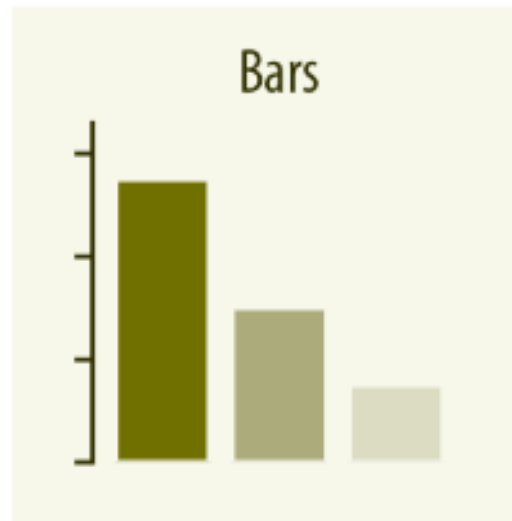
- **Heatmaps:** Represent the strength and direction of correlation between variables in a matrix format, with color coding to indicate the magnitude of correlation.
- Useful for identifying multicollinearity or strongly correlated variables in datasets.



Data Visualization - Types

3. **Categorical Data:** Bar charts, pie charts, count plots.

- **Pie Chart:** Display relationships in categorical data, such as market share or product category distribution. (Proportions/ percentages)

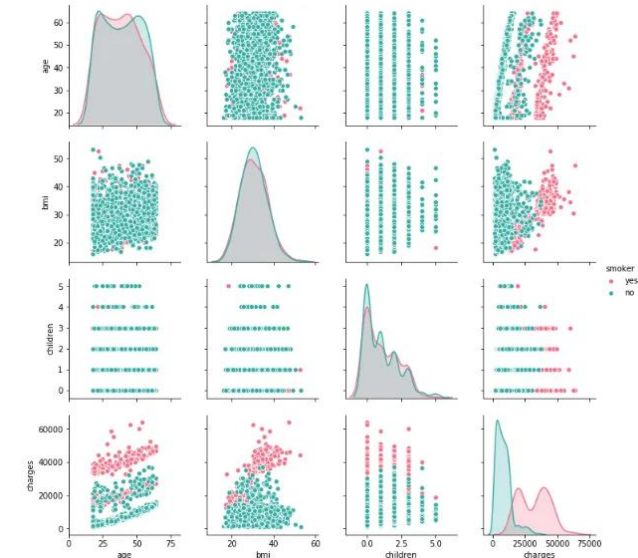
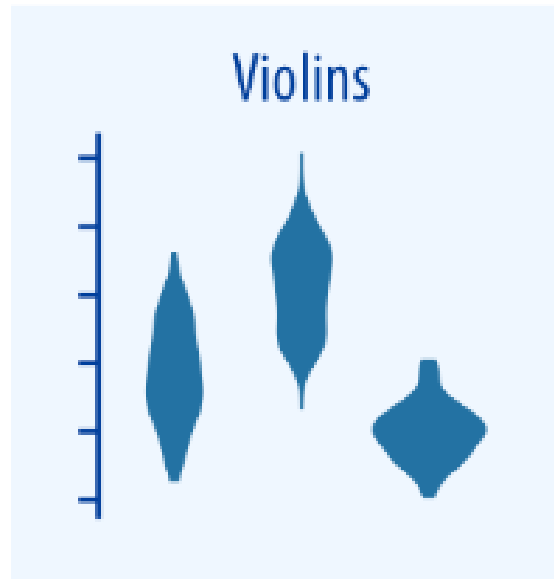
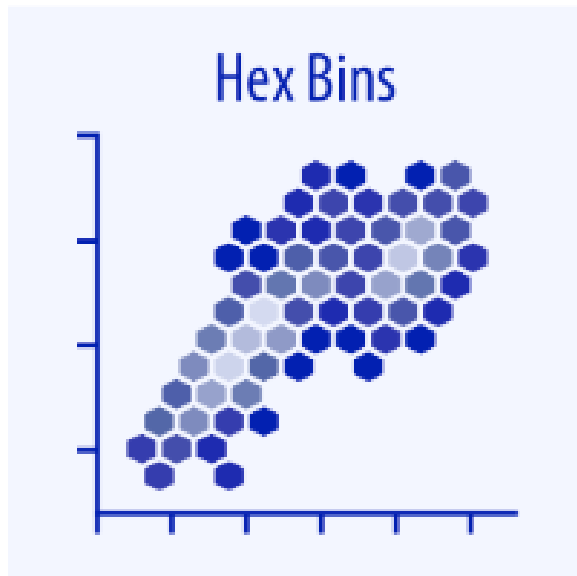


- **Bar Charts:** Show the frequency of different categories by displaying bars whose height represents the count of observations.

Data Visualization - Types

4. Bivariate and Multivariate: Hexbin plots, pair plots, and violin plots.

- **Hexbin Plots:** They are useful for very large datasets, where scatter plots might get cluttered.
- They group data into hexagonal bins and color them based on the number of observations.



Categorical Data - Summarization

Summarize and analyze Categorical data

1. **Frequency Distribution:** Represents a table or chart that displays how often each category occurs in the dataset
2. Useful for understanding the prevalence of **different categories**. Ex: Customer Types, Product Categories.
3. You can also visually represent the distribution of categories.
4. Ex: Use the Pandas **value_counts()** function to summarize categorical data and create a bar plot to visualize the frequency of each category.

Feature Engineering



Improve Model Performance

Handle Non-Linearity

Reduce Data Complexity (High-Dimensional)

Handle Categorical Data

Address Missing Values

Reduce Overfitting

Feature Engineering

The first step is **data exploration** which helps in feature engineering.

This involves understanding the data, identifying missing values, and checking for outliers.



Feature Extraction

Feature extraction involves selecting the most relevant features from the raw data.

Ex: Techniques include dimensionality reduction, principal component analysis, and clustering.



Feature Transformation

Feature transformation involves modifying the existing features to improve their predictive power.

Ex: Techniques include scaling, normalization, and logarithmic transformation.

Feature Engineering

Feature Creation

Feature creation involves generating new features from the existing ones.

Feature Selection

Feature selection involves choosing the most important features for the model.

Ex: Selection based on the **relationship** between **dependent** and **independent** variables.

These prevent the curse of dimensionality and improve the performance of the Model. It helps in visualizing and understanding the data.

Feature Engineering

Feature engineering (FE) aims to extract, select, or create relevant features from the raw data that can help the machine learning model better **understand the underlying patterns in the data**.

- By providing the model with **meaningful features**, it can **learn more efficiently** and make better predictions.
- **FE** allows us to transform the input features or create new features that capture **non-linear relationships**. This enables the model to capture more complex patterns in the data.
- **Imputation** is one of the FE approaches that can help **fill in missing values** using appropriate strategies, such as mean, median, or mode imputation, or more advanced techniques like predictive imputation.

Feature Engineering

- The **FE** techniques such as **one-hot encoding** or **ordinal encoding** can transform categorical variables into a format that the model can understand.
- It can help **prevent overfitting** by removing irrelevant features, **reducing noise** in the data, or applying regularization techniques to penalize overly complex models.
- By **simplifying** the model's **representation of the data**, it can improve its ability to generalize to unseen data.
- The real-world datasets contain a **large number of features**, some of which may be irrelevant or redundant. The FE techniques such as **dimensionality reduction** (e.g., PCA) can help **reduce the complexity of the data**, making it easier for the model to learn and generalize.

Feature Transform - Scaling (Numerical)

One of the most important transformations you need to apply to your data is feature scaling.

- With few exceptions, Machine Learning algorithms don't perform well when the input numerical attributes have **very different scales**.
- For example, in the housing dataset, the house prices range from **hundreds** to **millions**.
- Another example is the feature, total number of rooms ranges from about **6** to **39,320**, while the median incomes only range from **0** to **15**.
- Note that scaling the **Target values** is not required.
- There are two common ways to get all attributes to have the same scale: **min-max** scaling and **standardization**.

Feature Scaling

1. **Min-max scaling** (many people call this normalization) is quite simple: values are shifted and rescaled so that they end up ranging from **0** to **1**.
 - **Scikit-Learn** provides a transformer called **MinMaxScaler** for this.
 - It scales by **subtracting the min value** and **dividing by the max minus the min**.
2. **Standardization** is quite different. First, it subtracts the mean value (so standardized values always have a zero mean), and then it divides by the standard deviation so that the resulting distribution has unit variance.
 - Scikit-Learn provides a transformer called **StandardScaler** for standardization.
 - Unlike min-max scaling, **standardization does not bound values to a specific range**, which may be a problem for some algorithms
 - Example neural networks often expect an input value ranging from 0 to 1). However, standardization is much less affected by **outliers**.

Min-max scaler vs StandardScaler

MinMaxScale: More sensitive to outliers because the scale is heavily influenced by the extreme maximum and minimum values.

StandardScaler: Less sensitive to outliers since it focuses on standard deviation.

Another example, suppose a district had a median income equal to **100** (by mistake). Min-max scaling would then squeeze all the other values from 0–15 down to 0–0.15, whereas standardization would not be much affected.

As with all the transformations, it is important to fit the scalers to the **training data only**, **not to the full dataset** (including the **test set**).

Feature Transform - Encoding (Categorical)

Data Encoding is an important pre-processing step in Machine Learning.

- It refers to the process of converting categorical or textual data into numerical format so that it can be used as input for algorithms to process.
- The reason for encoding is that most **machine learning algorithms work with numbers** and not with text or categorical variables.
- This allows the model to **identify patterns** in the data and make predictions based on those patterns.
- Encoding also helps to **prevent bias** in the model by ensuring that all features are equally weighted.

There are two types of categorical data:

- **Ordinal data** Ex: Grades A, A-, B+, B, B- or one, two, three or A, B, C ...
- **Nominal data** Ex: NY, NJ, CT ...,

Feature Transform – One-hot Encoding (Categorical)

One-hot encoding is the Most Common method for encoding Categorical variables.

A Binary Column is created for each Unique Category in the variable.

If a category is present in a sample, the corresponding column is set to 1, and all other columns are set to 0.

For example, if a variable has three categories 'A', 'B' and 'C', three columns will be created and a sample with category 'B' will have the value [0,1,0].

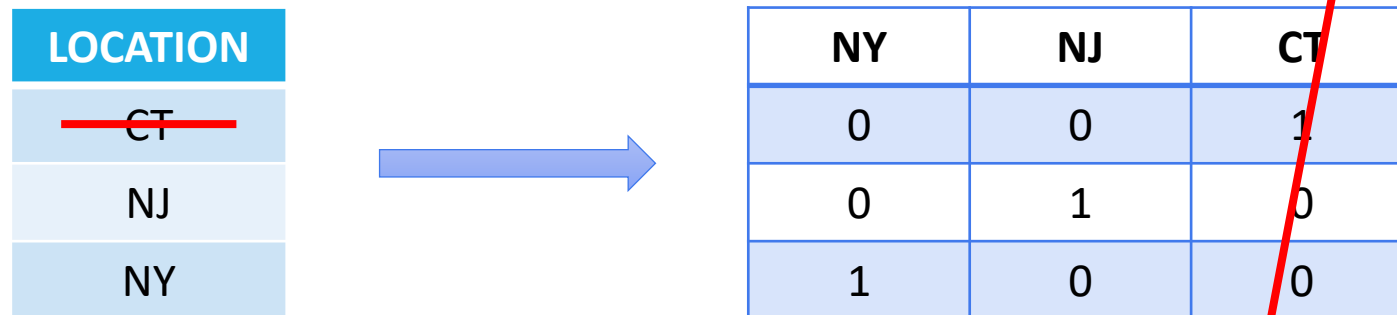
LOCATION		NY	NJ	CT
CT	→	0	0	1
NJ		0	1	0
NY		1	0	0

Feature Transform – Dummy Encoding (Categorical)

Dummy coding is similar to one-hot encoding.

This categorical data encoding method transforms the categorical variable into a set of binary variables [0/1].

The dummy encoding is a small improvement over one-hot encoding. Dummy encoding uses N-1 features to represent N labels/categories.



LOCATION
CT
NJ
NY

NY	NJ	CT
0	0	1
0	1	0
1	0	0

Feature Transform – Ordinal Encoding (Categorical)

Ordinal Encoding is used when the categories in a variable have a Natural Ordering.

In this method, the categories are assigned a numerical value based on their order, such as 1, 2, 3, etc.

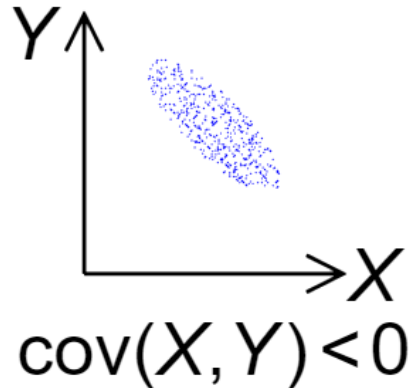
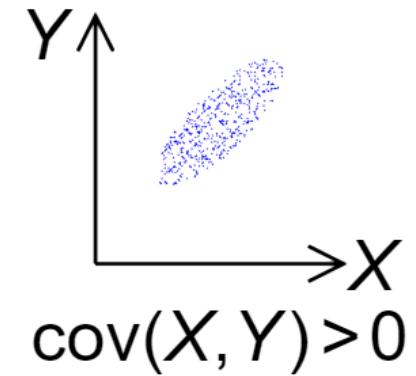
For example, if a variable has categories 'Low', 'Medium' and 'High', they can be assigned the values 1, 2, and 3, respectively.

Grades		Grades	Encoded
A		A	5
A-		A-	4
B+		B+	3
B-		B-	2
C		C	1
F		F	0

Feature Selection - Covariance

1. X increases y increases (linear). X decreases, y decreases.

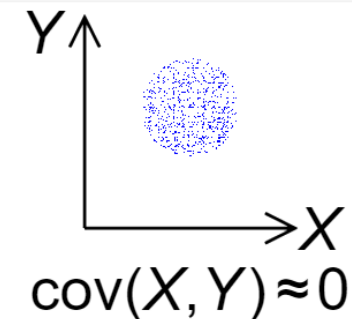
Positive relationship



2. X increases y decreases. X decreases y increases.

Negative relationship

3. X increases Y increases linear. **No relationship**



Feature Selection - Covariance

Population Covariance Formula

$$\text{Cov}(x, y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N}$$

Sample Covariance

$$\text{Cov}(x, y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N-1}$$

For a positive covariance value, we will have a **positive relationship** among the variables. If it approximates **0**, then there is **no relationship**.

High positive covariance indicates that the feature is most important.

Feature Engineering - Drawbacks

- Feature engineering can be time-consuming.
 - An efficient FE approach requires domain knowledge. SME dependent.
 - Overfitting and underfitting can occur if the wrong features are selected.
 - Understand requirements and rules before you adapt as different models may need different FE techniques.
 - FE can be an iterative process, requiring multiple iterations to achieve optimal results.
-

END
